Hive

Agenda

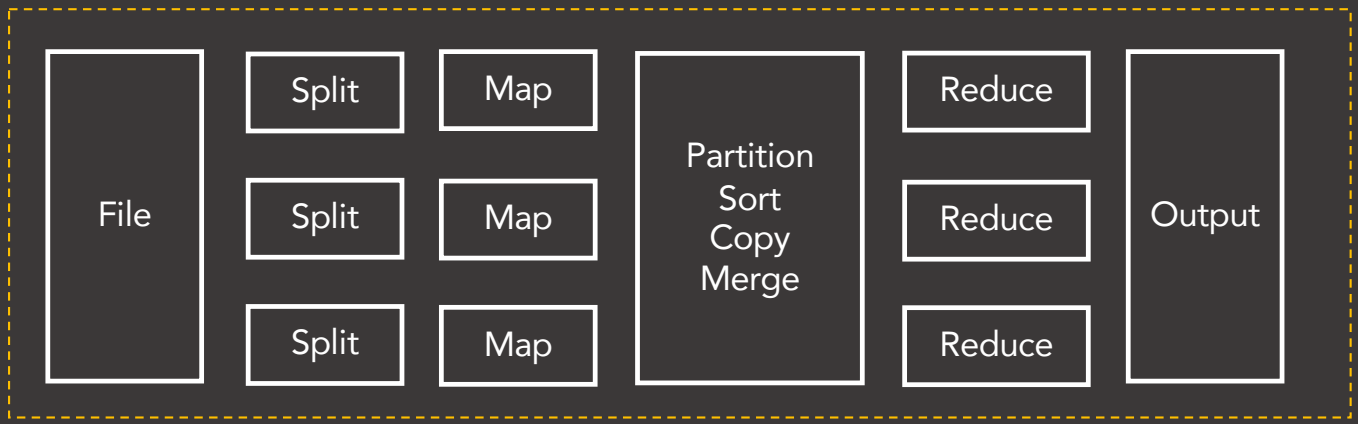
- Introduction to Hive
- Hive architecture & components
  - Metastore
  - Thrift
  - Hive clients
- Data types
- Hive tables
- File formats and record formats (SerDe)
- HiveQL – DDL
- HiveQL – DML

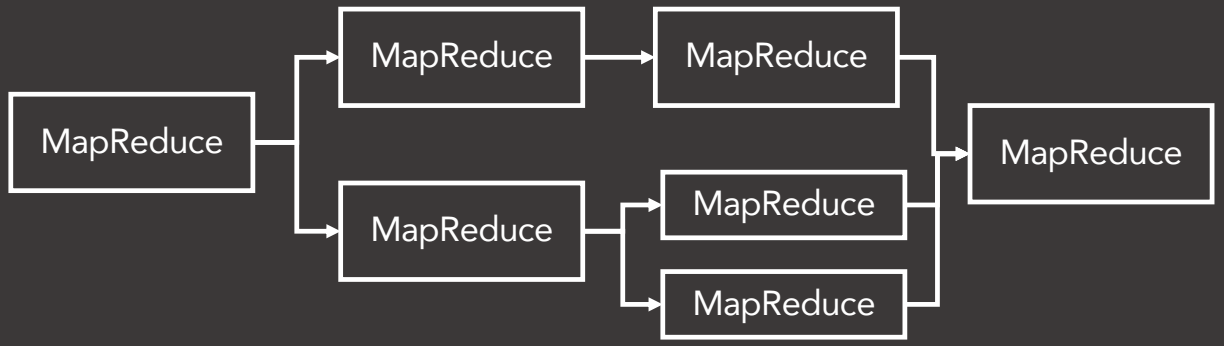
Agenda

- Partitioning & bucketing
- ACID transactions in Hive – UPDATE, DELETE
- Configuration
- Locks in Hive
- Views & indexes
- Storage handlers for NoSQL
- Hive integration with other frameworks

Introduction

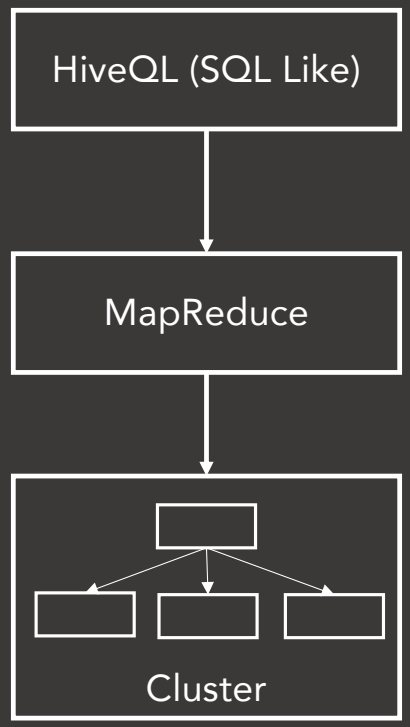
- MapReduce
  - Mapper
  - Reducer
- SQL





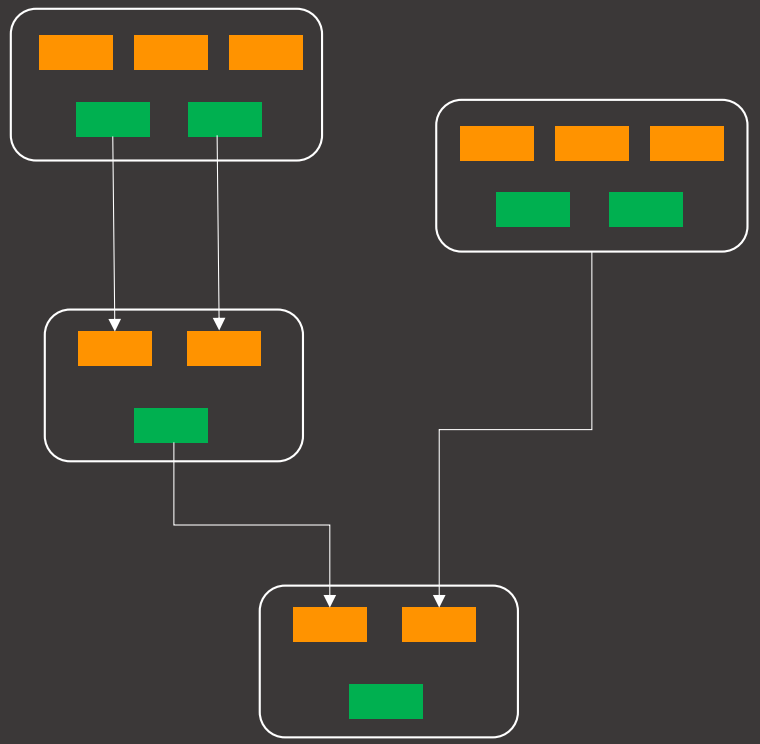
Introduction

- Hive was built on top of Hadoop for querying, summarizing, and analyzing large data sets using a SQL-like interface.
- Hive Query Language (HiveQL).
- It is noted for bringing the familiarity of relational technology to Big Data processing with Hive Query Language.
- HiveQL gets converted to MapReduce jobs and runs on Hadoop.
- MapReduce is slow.



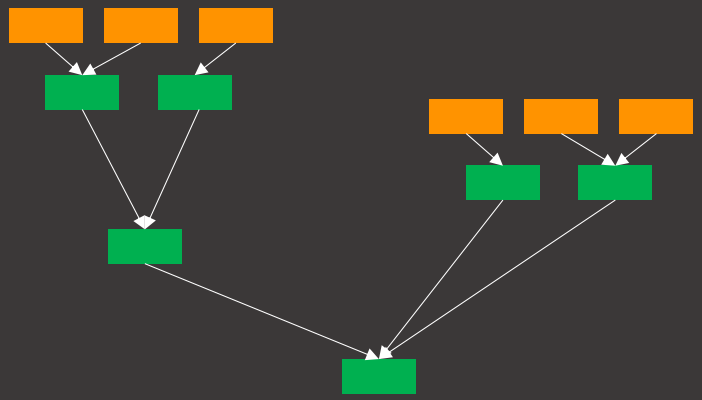
Tez

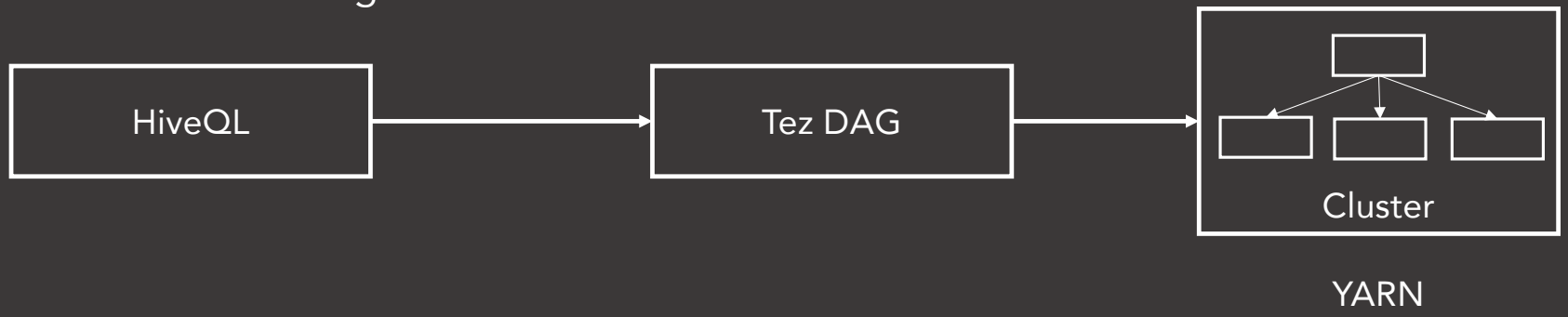
- Execution engine for Hive. Replacement of MapReduce.
- Complex flow of MapReduce jobs



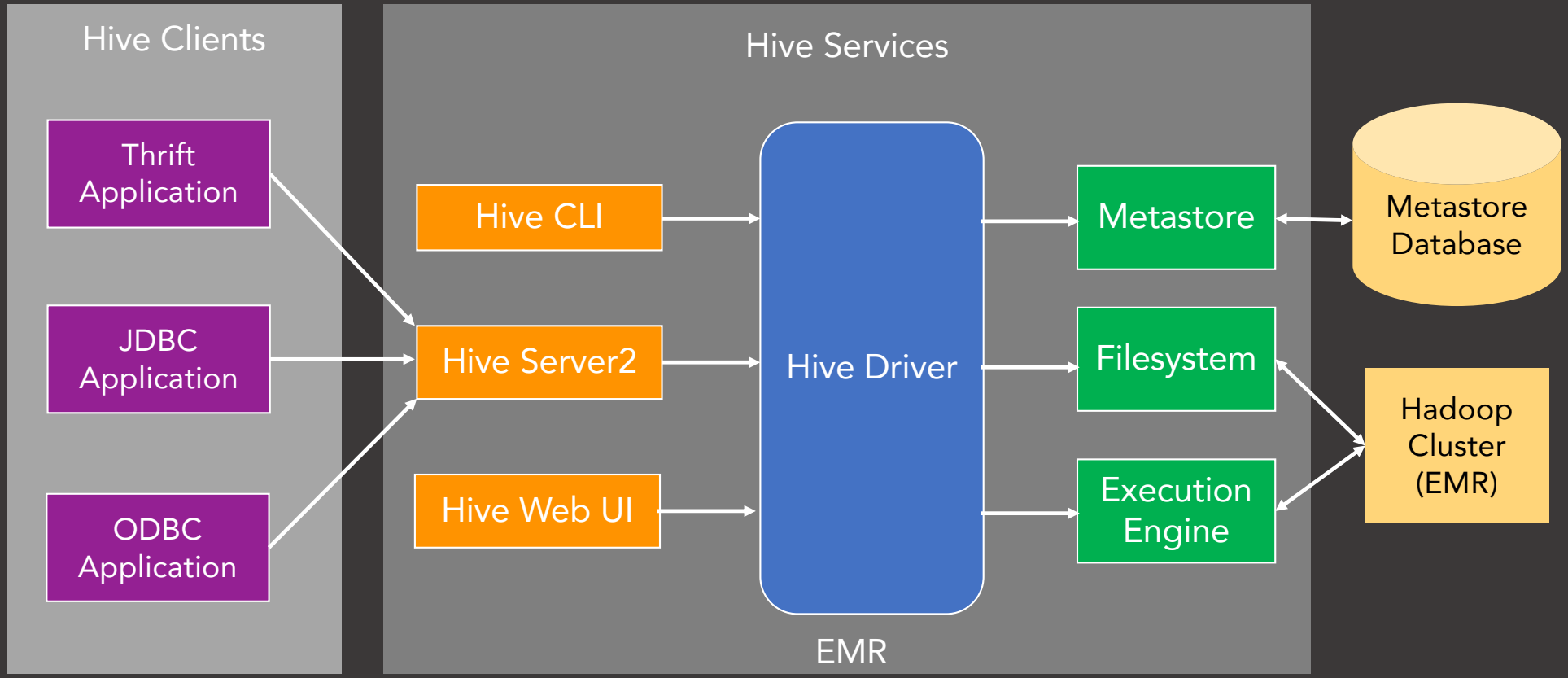
Tez

- Multiple MapReduce jobs can be replaced by single Tez job.
- Directed Acyclic Graph (DAG)
- Hive queries gets converted to Tez DAGs.
- Hive in EMR runs on Tez.
- hive.execution.engine.





Hive Architecture



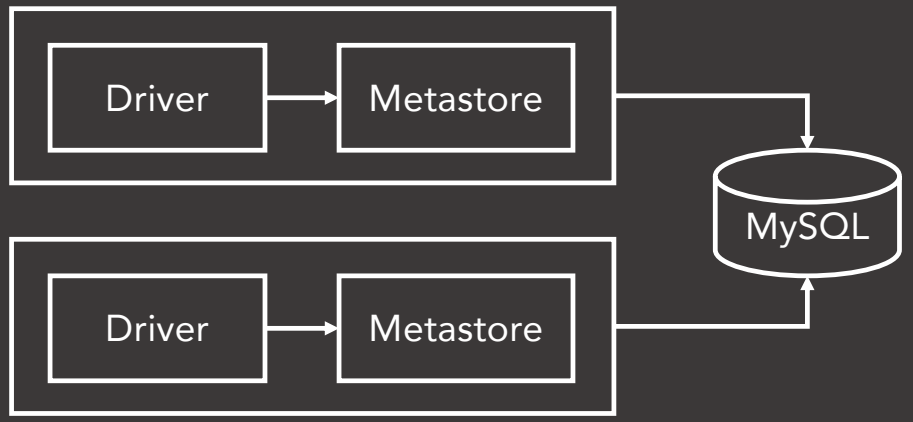
Hive Components

- Hiveserver2 – Runs Hive as a server exposing Thrift service. Enables access from programming languages that support Thrift, JDBC and ODBC.
- CLI – Command line interface for Hive. Hive shell.
- Web UI – Alternative of Hive CLI. It provides a web-based GUI for executing Hive queries and commands.
- Hive Metastore (HMS) – Stores all the structure information of various tables and partitions.
- Execution Engine - Optimizer generates logical plan in the form of DAG of Tez tasks or MapReduce tasks and HDFS tasks.
- Filesystem – Communicates with various file formats and record formats.
- Beeline – JDBC based command line shell.

Hive Metastore (HMS)

- Why do we need metastore?
- Central repository for Hive metadata.
- Two components:
  - Service
  - Data storage
- Types of metastores
  - Embedded metastore – Derby
  - Local metastore

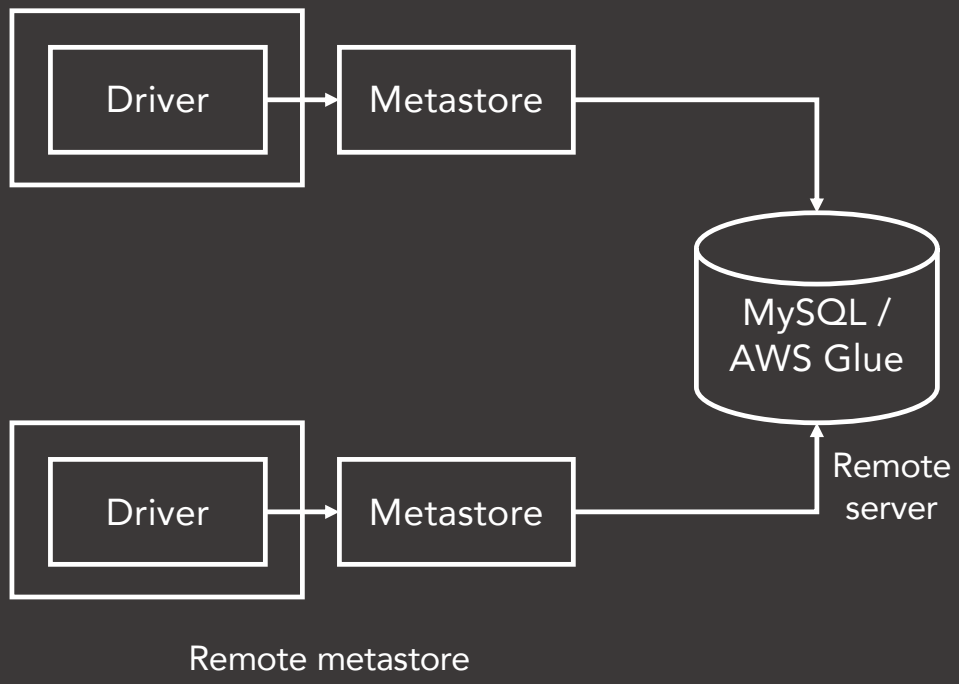
Embedded metastore



Local metastore

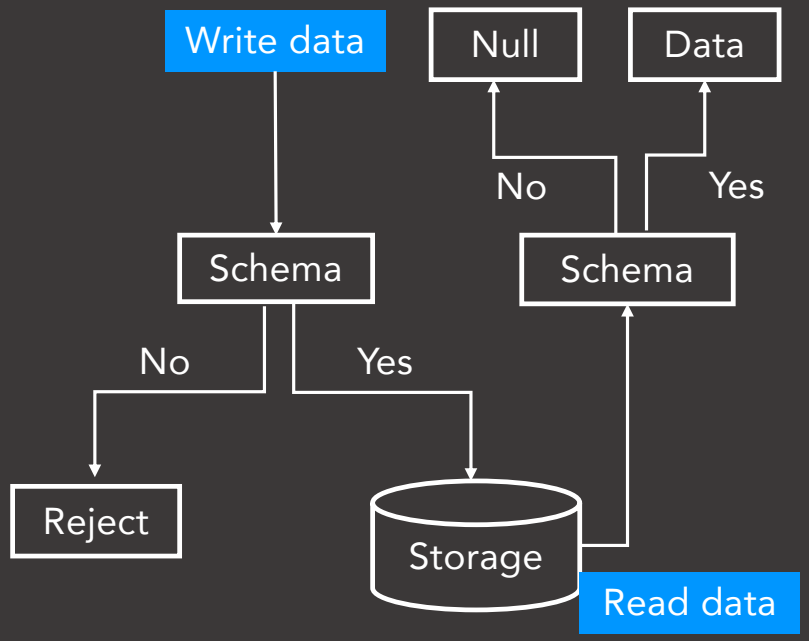
Hive Metastore (HMS)

- Why do we need metastore?
- Central repository for Hive metadata.
- Two components:
  - Service
  - Data storage
- Types of metastores
  - Embedded metastore – Derby
  - Local metastore
  - Remote metastore



Schema on Read

- Traditional database (schema on write)
  - Schema is enforced at write time.
  - If data does not conform to schema, it is rejected.
- Hive (schema on read)
  - Data is not verified when loaded, but when it is read.
  - If data does not conform to schema while reading, NULL is returned.



Hive CLI on EMR

Data Types

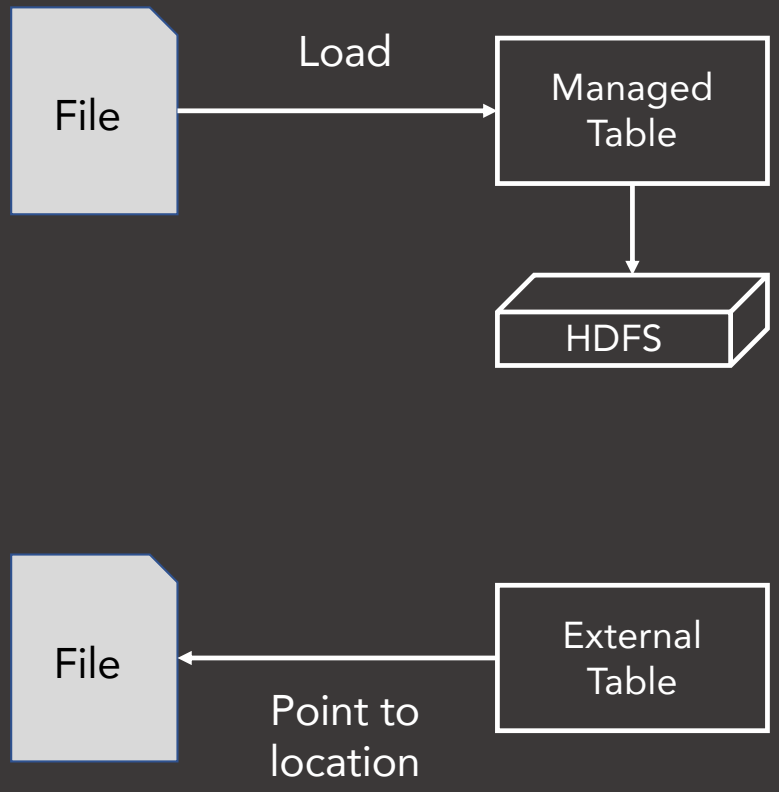
- Numeric type
  - TINYINT, SMALLINT, INT, BIGINT
  - FLOAT, DOUBLE
  - DECIMAL(p, s), NUMERIC(p, s)
- Date/Time type
  - TIMESTAMP – YYYY-MM-DD HH:MM:SS.fffffffff
  - DATE – YYYY-MM-DD
  - INTERVAL
- String type
  - STRING
  - VARCHAR(n = 1-65535)
  - CHAR(n = 1-255)

Data Types

- Complex type
  - Arrays - ARRAY<data_type>.
  - Maps - MAP<col_name, data_type, … >.
  - STRUCT<col_name: data_type, col_name: data_type … >

Databases & Tables

- Database
- Table
  - CREATE TABLE
  - Managed table – Data lifecycle is managed by Hive.
  - External table – Hive does not own the data.
  - Temporary table – Tables are visible to the current session only.



File/Storage Format

CREATE TABLE … STORED AS <file_format> … ROW FORMAT <row_format>

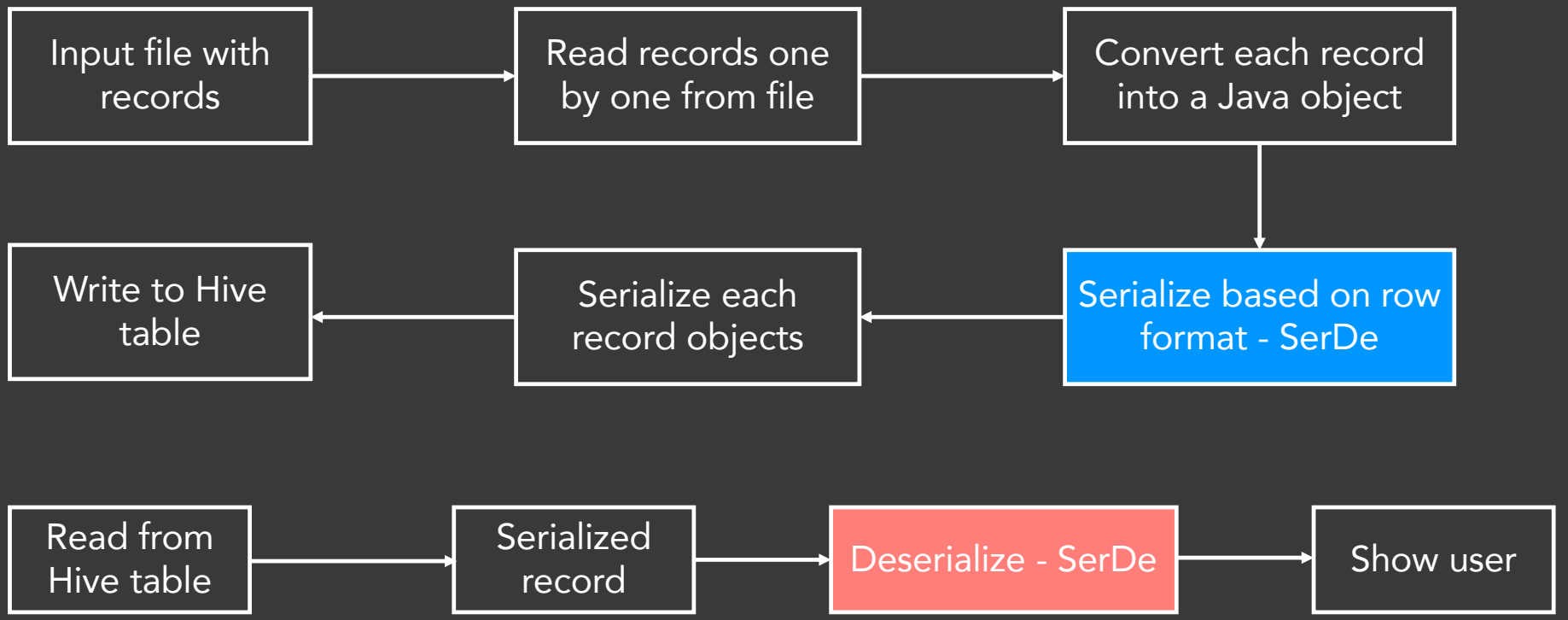
- Supported file formats
  - Text File
  - SequenceFile
  - Parquet
  - Avro
  - ORC
  - RCFile
  - Custom file formats using INPUTFORMAT and OUTPUTFORMAT

File/Storage Format

CREATE TABLE … STORED AS <file_format> … ROW FORMAT <row_format>

- SEQUENCEFILE – Stored as compressed Sequence File.
- TEXTFILE – Stored as plain text file.
- RCFILE – Stored as record columnar file format.
- ORC – Stored as ORC format. Supports ACID transactions.
- PARQUET – Stored as Parquet columnar format.
- AVRO – Stored as Avro file format.
- JSONFILE – Stored as JSON files.
- INPUTFORMAT input_format_classname OUTPUTFORMAT output_format_classname - Custom file format.

Record/Row Format (SerDe)



Record/Row Format (SerDe)

- Native SerDe
  - “STORED AS” does not need explicit row format to be mentioned.
  - Hive SerDe library - org.apache.hadoop.hive.serde2
  - JSON - ROW FORMAT SERDE 'org.apache.hive.hcatalog.data.JsonSerDe' STORED AS TEXTFILE
  - CSV - ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde' STORED AS TEXTFILE
- Custom SerDe
- Stored as parquet, rcfile, orc, avro, sequencefile does not need ROW FORMAT to be mentioned. Row format is controlled by the file format itself.

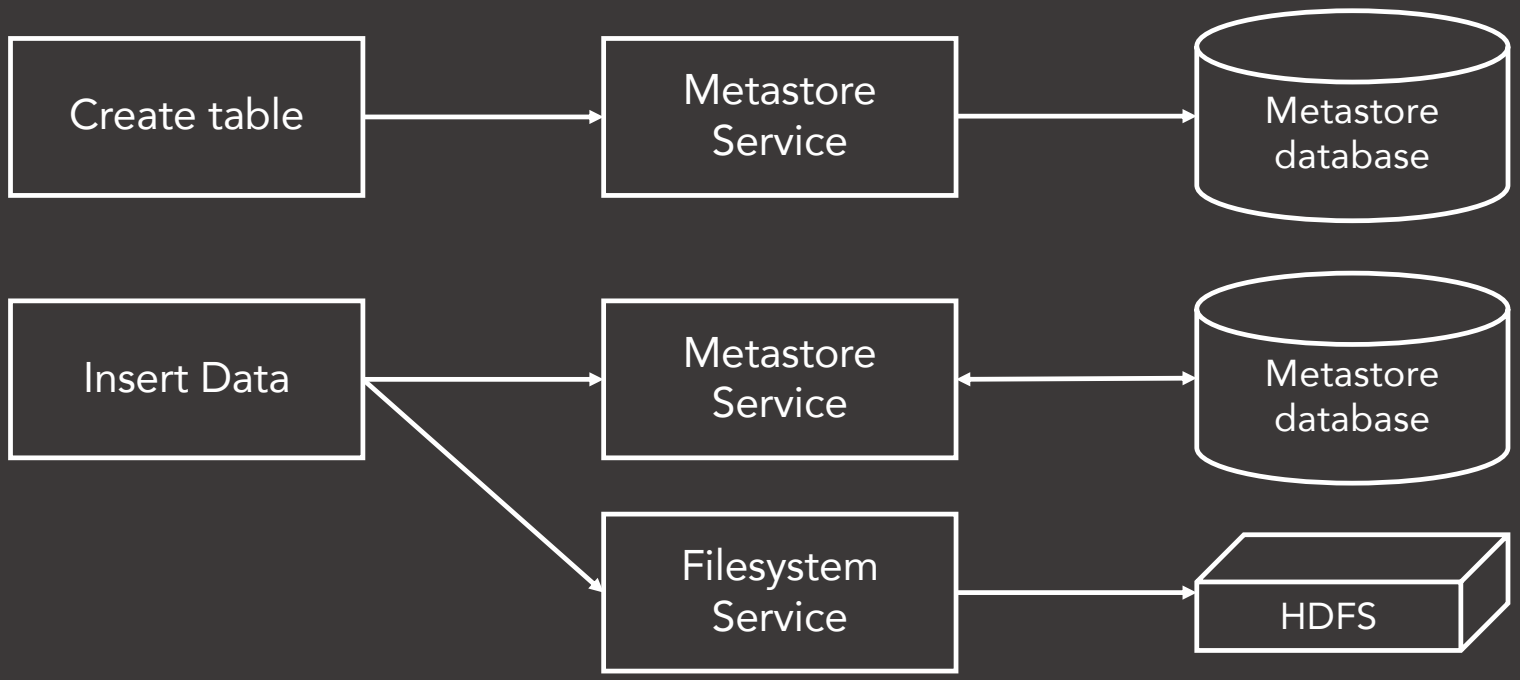
Record/Row Format (SerDe)

```
CREATE TABLE de_test
ROW FORMAT SERDE ‘org.apache.hadoop.hive.serde2.avro.AvroSerDe’
STORED AS
INPUTFORMAT ‘org.apache.hadoop.hive.ql.io.avro.AvroContainerInputFormat’
OUTPUTFORMAT ‘org.apache.hadoop.hive.ql.io.avro.AvroContainerOutputFormat’
```

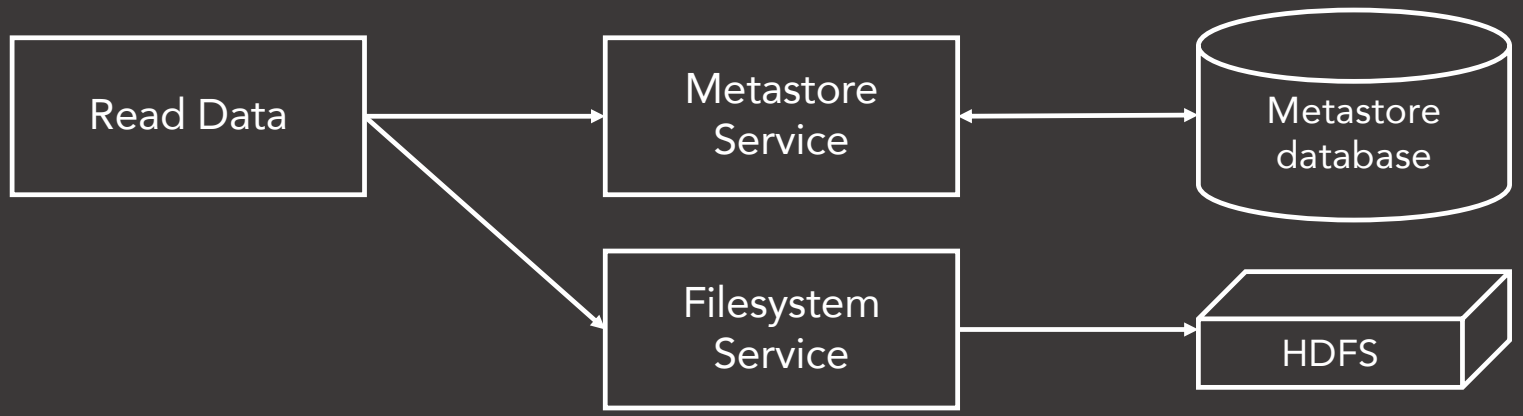
Create Table

- CREATE TABLE table_name
- CREATE EXTERNAL TABLE table_name
- CREATE TEMPORARY TABLE table_name
- STORED AS file_format
- ROW FORMATE SERDE
- LOCATION distributed_storage_path
- TBLPROPERTIES (…)
- PRIMARY KEY (col_name, …)

Hive Write Path – Service view



Hive Read Path – Service view



Hive hands-on

Table Partitioning

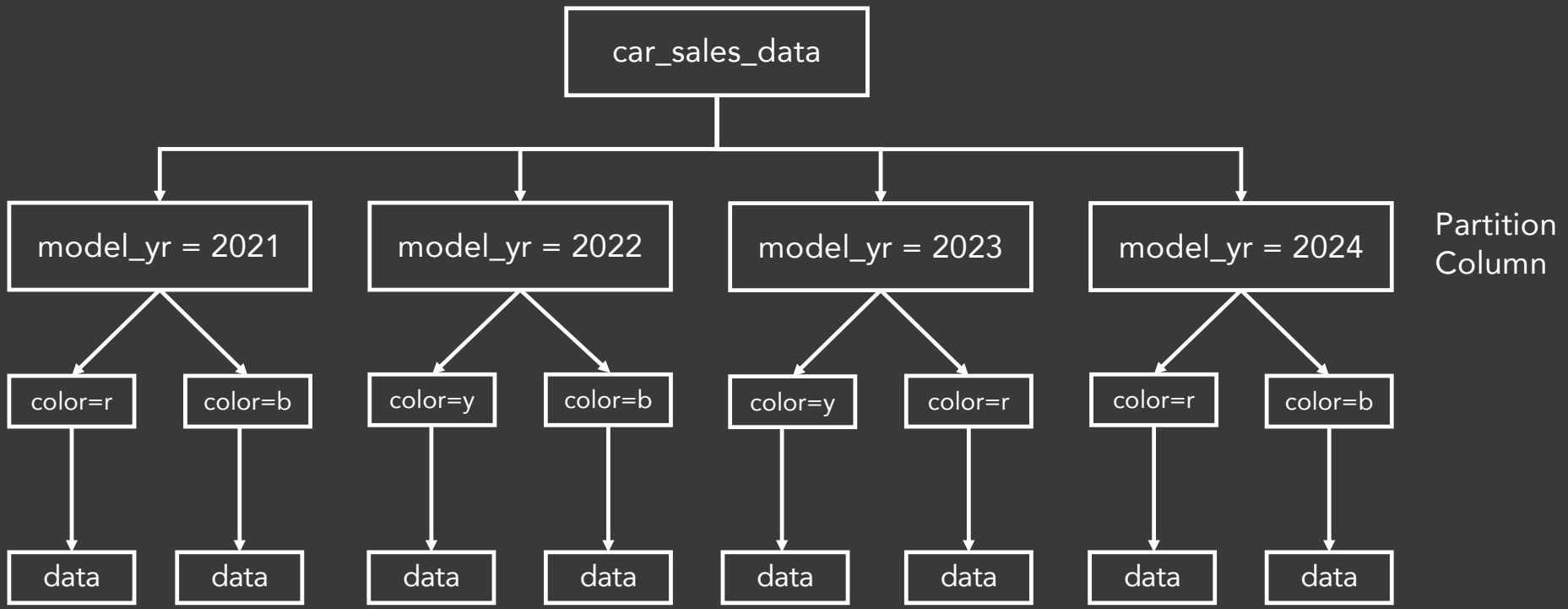


Table Partitioning – Managed Tables

- Store different data (based on columns) in different locations.
- Separate data directory is created for each distinct value combination in the partition columns.
- Faster queries by partition pruning.
- WHERE clause should contain the partition column. Partition Filters.

```
CREATE TABLE table_name ( ... )
PARTITIONED BY (col_name data_type, col_name data_type, ... );

ALTER TABLE table_name ADD PARTITION (part_col = value, part_col = value, … );
SHOW PARTITIONS table_name;
DESCRIBE EXTENDED table_name;
```

hive.mapred.mode=strict|nonstrict

Table Partitioning – External Tables

```
CREATE EXTERNAL TABLE table_name ( ... )
PARTITIONED BY (col_name data_type, col_name data_type, ... );
```

- LOCATION clause is not needed while defining the table.
- ALTER TABLE statement is used to add each partition separately
- Value for each partition key must be specified.

```
ALTER TABLE table_name ADD PARTITION (part_col = value, part_col = value, … )
LOCATION ‘dir_location’;
```

Table Partitioning - Static

- Static partitioning.
- Manual partitioning.
- Data files are loaded individually for each partition.
- Correct data has to be loaded in correct partition. <file> should contain data corresponding to that partition.
- Takes less time to load.
- **`hive.mapred.mode`** has to be “strict”.
- WHERE clause should be used to limit the partition.

```
CREATE EXTERNAL TABLE <table_name> ( ... )
PARTITIONED BY (model_yr INT)
...;

LOAD DATA ... <file> INTO TABLE
car_sales_data PARTITION (model_yr=2023);

LOAD DATA ... <file> INTO TABLE
car_sales_data PARTITION (model_yr=2022);
```

Table Partitioning - Dynamic

- set hive.exec.dynamic.partition=true;
- set hive.exec.dynamic.partition.mode=nonstrict;
- Partitions are created on the fly.
- INSERT command can be used to load data, which will be put into their corresponding partition automatically.
- Takes more time to load.

```
CREATE EXTERNAL TABLE <table_name> ( ... )
PARTITIONED BY (model_yr INT)
...;

INSERT INTO table PARTITION (column) ...
```

Table Partitioning - Dynamic

- `hive.exec.dynamic.partition = true`
  - Needs to be set to true to enable dynamic partition inserts.
- `hive.exec.dynamic.partition.mode = strict|nonstrict`
  - In strict mode, the user must specify at least one static partition in case the user accidentally overwrites all partitions, in nonstrict mode all partitions are allowed to be dynamic.
- `hive.exec.max.dynamic.partitions = n`
  - Maximum number of dynamic partitions allowed to be created in total.
- `hive.error.on.empty.partition = true|false`
  - Whether to throw an exception if dynamic partition insert generates empty results.

Bucketing

- Buckets divides tables or partitions into further sub-divisions.

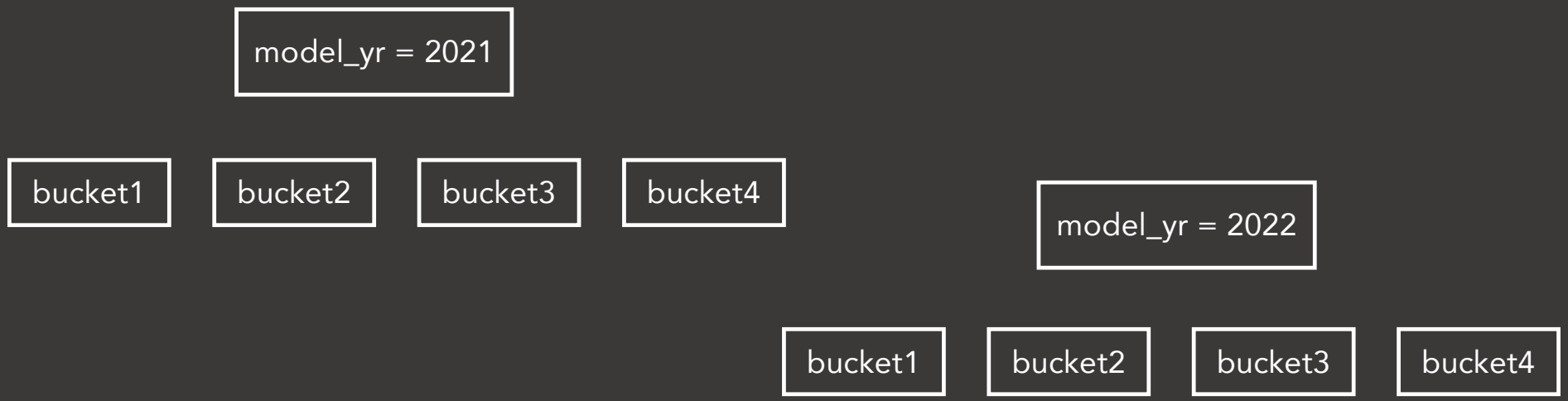
```
CREATE TABLE table_name (product_id INT, sales_person_id STRING, ... )
CLUSTERED BY (product_id) INTO 4 BUCKETS;
```

```
CREATE TABLE table_name (product_id INT, sales_person_id STRING, ... )
PARTITIONED BY(model_year)
CLUSTERED BY (product_id) INTO 4 BUCKETS;
```

- Bucket No. = hash(clustered_col) % No. of partitions.
- Bucket No. = hash(product_id) % 4.

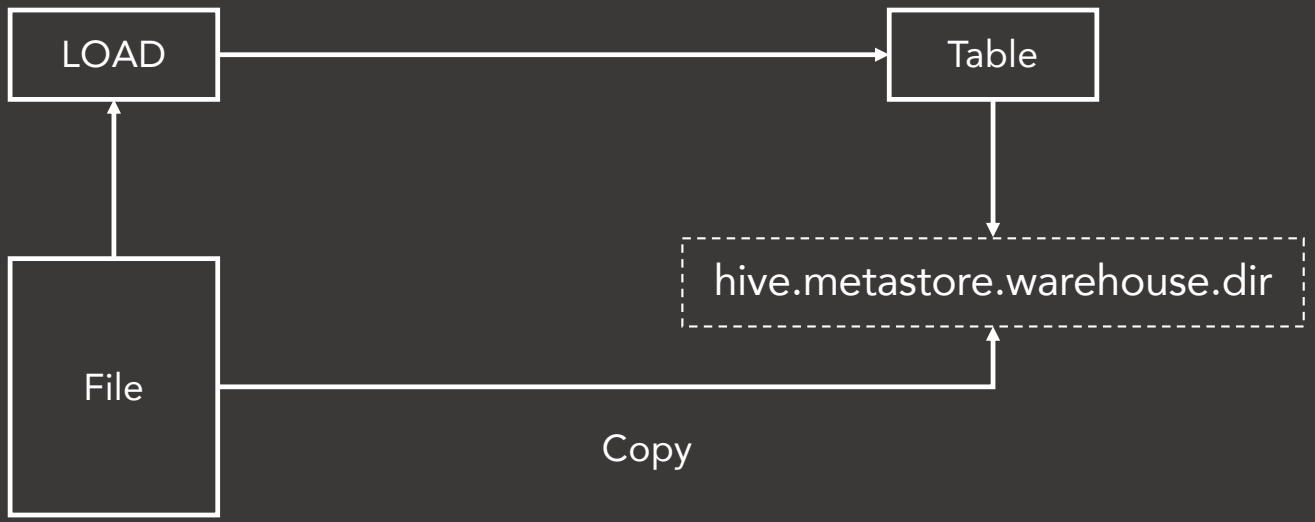
Bucketing

```
CLUSTERED BY (product_id) INTO 4 BUCKETS;
```



Partitioning & Bucketing hands-on

LOAD



- No Transformation
- Direct Copy

INSERT

- INSERT INTO ... – Append to the table.
- INSERT OVERWRITE … - Overwrite existing data.
- Table or Partition
- Transformation
- Read from CSV and insert into ORC, Parquet etc.
- INSERT INTO … SELECT … FROM …

SELECT

```
[WITH CommonTableExpression (, CommonTableExpression)]
SELECT [ALL | DISTINCT] select_expr, select_expr, ...
  FROM table_reference
  [WHERE where_condition]
  [GROUP BY col_list]
  [ORDER BY col_list]
  [CLUSTER BY col_list
    | [DISTRIBUTE BY col_list] [SORT BY col_list]
  ]
 [LIMIT [offset,] rows]
```

- ORDER BY – Order is maintained for the entire result set.
- SORT BY – Ordering rows within a reducer. Partially ordered dataset.
- Local Mode – Read from files directly.

ACID Transactions - UPDATE, DELETE

- UPDATE & DELETE statements.
- BEGIN, COMMIT and ROLLBACK are not supported.
- Only ORC file format is supported.
- Must be bucketed.
- Have to set Hive transaction manager.
  - `SET hive.txn.manager = org.apache.hadoop.hive.ql.lockmgr.DbTxnManager;`
  - `SET hive.support.concurrency = true;`
  - `SET hive.enforce.bucketing = true;`
- `CREATE TABLE … TBLPROPERTIES (“transactional”=“true”);`

ACID Transactions, UPDATE, DELETE

- Base Files
- Delta Files
- Compaction
  - Minor compaction – Takes a set of existing delta files and rewrites them to a single delta file per bucket.
  - Major compaction – Takes one or more delta files and the base file for the bucket and rewrites them into a new base file per bucket.

Materialized View

- Similar to a table.
- Stores results of a query.
- Refresh the content of MV to keep it up-to-date with the changes in the table.
- Refresh is incremental by default, falls back to rebuild if not possible.
  - INSERT – Incremental
  - UPDATE – Full rebuild
  - DELETE – Full rebuild

```
CREATE MATERIALIZED VIEW <name> …
PARTITIONED ON (col_name, ...)
CLUSTERED ON (col_name, ...)
ROW FORMAT <row_format>
FILE FORMAT <file_format>
LOCATION <location>
AS <query>
```

```
ALTER MATERIALIZED VIEW <name> REBUILD;
```

Index

- Indexes are removed since 3.0

View

- Similar to view in RDBMS.
- CREATE VIEW AS SELECT …

JOINs

- INNER JOIN
- LEFT | RIGHT OUTER JOIN

```
SELECT a.*
FROM a JOIN b ON (a.id = b.id AND a.department = b.department)

SELECT a.* FROM a LEFT OUTER JOIN b ON (a.id = b.id)

SELECT a.val, b.val, c.val
FROM   a JOIN b ON (a.key = b.key1)
         JOIN c ON (c.key = b.key2)
```

Hive Configuration Parameters

- hive.mapred.mode=strict - Full table scans are rejected and ORDER BY needs a LIMIT clause.
- hive.exec.parallel=true|false - Whether to execute jobs in parallel. Applies to MapReduce jobs that can run in parallel.
- hive.exec.parallel.thread.number=8 - How many jobs at most can be executed in parallel.
- hive.enforce.bucketing=true - While inserting into the table, bucketing is enforced.
- hive.enforce.sorting=true - while inserting into the table, sorting is enforced.
- hive.exec.dynamic.partition=true – Enable dynamic partitioning.
- hive.exec.dynamic.partition.mode=strict
- hive.exec.mode.local.auto=true – Whether local mode should be used in SELECT.

Hive Configuration Parameters

- hive.exec.temporary.table.storage - Define the storage policy for temporary tables.
- hive.default.serde=org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe - The default SerDe Hive will use for storage formats that do not specify a SerDe (TextFile, RCFile).
- hive.default.fileformat=TextFile - Default file format for CREATE TABLE statement. Options are TextFile, SequenceFile, RCfile, ORC, and Parquet.
- hive.metastore.warehouse.dir=/user/hive/warehouse - Location of default database for the warehouse.
- hive.execution.engine=mr | tez | spark
- mapred.reduce.tasks - The default number of reduce tasks per job.
- hive.merge.tezfiles=true - Merge small files at the end of a Tez DAG.

Hive Configuration Parameters

- hive.txn.manager=org.apache.hadoop.hive.ql.lockmgr.DbTxnManager - Turns on Hive transactions.
- hive.compactor.initiator.on=true
- hive.compactor.cleaner.on=true
- hive.compactor.worker.threads=4 - How many compactor worker threads to run on this metastore instance.
- hive.support.concurrency=true - Whether Hive supports concurrency or not.
  A ZooKeeper instance must be up and running for the default Hive lock manager to support read-write locks.
- hive.enforce.bucketing=true
- hive.exec.dynamic.partition.mode=nonstrict
- hive.lock.manager=org.apache.hadoop.hive.ql.lockmgr.zookeeper.ZooKeeperHiveLockManager

Hive Configuration Parameters

- hive.hwi.war.file
- hive.hwi.listen.host
- hive.hwi.listen.port

- File where configuration parameters are stored – hive-site.xml

Configuration on EMR

- By supplying configuration object (JSON file).
- Classification and properties.
- Classification maps to application specific configuration file.
  - “hive-site” classification maps to “hive-site.xml” configuration file.

```
[
  {
    "Classification": "core-site",
    "Properties": {
      "hadoop.security.groups.cache.secs": "250"
    }
  },
  {
    "Classification": "mapred-site",
    "Properties": {
      "mapred.tasktracker.map.tasks.maximum": "2",
      "mapreduce.map.sort.spill.percent": "0.90",
      "mapreduce.tasktracker.reduce.tasks.maximum": "5"
    }
  }
]
```

EXPLAIN Statement

- Check how Hive will execute a query.
- EXPLAIN [EXTENDED | CBO | AST | DEPENDENCY | AUTHORIZATION | LOCKS | VECTORIZATION | ANALYZE] <query>
- Query gets converted into a sequence of stages.
  - Map/Reduce stages
  - Metastore operations
  - Filesystem operations
- There are dependencies between different stages of the plan.

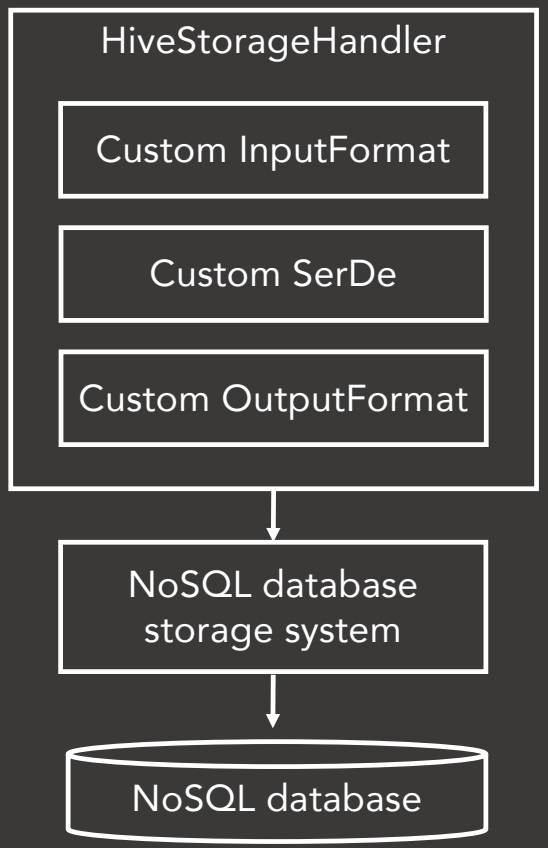
Lock Modes

- Shared – While reading
- Exclusive – All other operations

Lock Compatibility		Existing Lock	
		S	X
Requested Lock	S	True	False
	X	False	False

Hive Command	Locks Acquired
select .. T1 partition P1	S on T1, T1.P1
insert into T2(partition P2) select .. T1 partition P1	S on T2, T1, T1.P1 and X on T2.P2
insert into T2(partition P.Q) select .. T1 partition P1	S on T2, T2.P, T1, T1.P1 and X on T2.P.Q
alter table T1 rename T2	X on T1
alter table T1 add cols	X on T1
alter table T1 replace cols	X on T1
alter table T1 change cols	X on T1
alter table T1 concatenate	X on T1
alter table T1 add partition P1	S on T1, X on T1.P1
alter table T1 drop partition P1	S on T1, X on T1.P1
alter table T1 touch partition P1	S on T1, X on T1.P1
alter table T1 set serdeproperties	S on T1
alter table T1 set serializer	S on T1
alter table T1 set file format	S on T1
alter table T1 set tblproperties	X on T1
alter table T1 partition P1 concatenate	X on T1.P1
drop table T1	X on T1

Storage Handlers



- Used to connect to NoSQL databases like HBase, Cassandra, DynamoDB etc.
- STORED BY HBaseStorageHandler. TBLPROPERTIES
- External Table
- STORED BY CassandraStorageHandler. SERDEPROPERTIES (column mappings).
- STORED BY DynamoDBStorageHandler. Column mapping is done is TBLPROPERTIES.

Hive Integrations

- HBase
- Cassandra
- DynamoDB
- Druid
- Kudu
- Spark

Thank You!